```
Enter total number of frames
Enter sender's window size : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss): 5

-----------------------------------------

Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.

-----------------------------------------

Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
    (Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 5.

-----------------------------------------

Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

-----------------------------------------

All 8 frames successfully transmitted from Node A to Node B.
```

.NO:7

ATE:- 06.08.26

FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

AIM:-

To write a program to implement flow control at the data Link layer using the Sliding window Protocol, and to Simulate the flow of frames from one node to another.

ALGORITHM:-

1. Start the program.
2. Read the total number of frames to be Send (N), the Sender's window Size (W), and (optionally) a frame number to Simulate as lost during transmission.
3. Initialize base = 0 (Sequence number of the oldest unacknowledged frame) and nextSeq = 0 (Sequence number of the next frame to be Sent).
4. Repeat while base < N:

a) Determine the current sender window as the range [base, base+W-1].

b) Send all frames with Sequence numbers from nextSeq up to (base+W-1), incrementing nextSeq after each frame, Since the window restricts the Sender to at most W outstanding frames at any time.

c) If the lost frame lies within the frames just Sent:

i) The receiver discards the lost frame and any subsequent out-of-order frames.

d) Else (no loss in this batch):

i) The receiver accepts each frame in order and sends a cumulative acknowledgement (ACK) for it.

5. Repeat step 4 until all N frames have been transmitted and acknowledge.

6. Stop the program.

PROGRAM :-

```
def Simulate (total_frames, window_size, last_frame):
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min (base + window_size - 1, total_frames - 1)
        print ("\n - - - - - - - - - - - - - - - ")
        print (f "Sender Window : [Frame {base} - Frame {window_
                                                    end}] ")
        # Send all frames within the current window that haven't been
          sent
        while next_seq < base + window_size and next_seq < total_
          frames :
            print (f "Node_Seq < base + window_size and next_seq <
            total_frames :
            if next_seq == last_frame :
                print (f " (Frame {next_seq} LOST in transit!) ")
                next_seq += 1
        if last_frame is not None and base <= last_frame < next_seq :
            # a frame in this batch was lost : receiver discards it and
            # everything after it (GO-BACK-N); Sender times out and
            # resends starting
```

```
print(f"Node B -->Frame {last_frame} missing. Discarding out-of-order frames after it.")
print(f"Node A -->Timeout! Resending window starting from Frame {last_frame}.")
next_seq = last_frame
last_frame = None # loss already handled; retransmission succeeds continue.
#for f in range(base, next_seq):
    print(f"Node B -->Received Frame {f}, sending ACK {f}")
print(f"Node A -->ACK {next_seq - 1} received. Sliding window forward.")
base = next_seq
print("\n" + "-" * 40)
print(f"All total_frames} frames successfully transmitted from Node A to Node B.")

def main():
print("=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===\n")
total_frames = int(input("Enter total number of frames to Send: "))
window_size = int(input(f"Enter frame number to Simulate as LOST\n(Out of {total_frames-1} enter -1 for no loss): "))
if last_frame == -1:
    last_frame = None
Simulate(total_frame, window_size, last_frame)
if __name__ == "__main__":
main()
```

OUTPUT :-

Enter total number of frames to send : 8

Enter Sender's window size : 4

Enter frame number to Simulate as LOST
(0 to 7, enter -1 for no loss) : 5

- - - - - - - - - -

Sender window : [Frame 0 - frame 3]

Node A -> Sending frame 0

Node A -> Sending frame 1

Node A -> Sending frame 2

Node A -> Sending frame 3

Node B -> Received frame 0 , Sending ACK 0

Node B -> Received frame 1 , Sending Ack 1

Node B -> Received frame 2 , Sending Ack 2

Node B -> Received frame 3 , Sending ACK 3

Node A -> ACK 3 received, Sliding window forward

Sender window : [frame 4 - frame 7]

Node A -> Sending frame 4

Node A -> Sending frame 5

(frame 5 LOST in transit :)

Node A -> Sending frame 6

Node -> Sending frame 7

Node -> frame 5 missing :(.

RESULT:-

The program to implement flow control at the Data Link layer using the Sliding window Protocol was successfully written in Python, executed and Verified by Simulating the flow of frames between two node (Node A as sender and Node B as receiver).